Foresight 프로젝트 - 기술 명세서

작성일: 2026년 4월
대상 버전: 현재 메인 브랜치
범위: Opening Data 수집/티어 선정, 게임 분석, 유저 분석, DB 구조

목차

1. Opening Data 수집/티어 선정 로직
2. 전적 검색의 게임 분석 로직
3. 전적 검색의 유저 분석 로직
4. 데이터베이스 구조

---

1. Opening Data 수집/티어 선정 로직

1.1 개요

Foresight는 **Lichess Explorer API**를 통해 전 세계 온라인 게임 통계를 수집하여, 각 오프닝별로 **S/A/B/C/D 티어**를 자동으로 배정합니다. 이는 체스 플레이어가 "이 오프닝이 강한인가?"를 빠르게 판단할 수 있도록 돕습니다.

핵심 특징:

- 카탈로그 기반 자동 탐색 (60+ ECO 오프닝)
- 월 1회 자정 UTC 자동 갱신 (전월 데이터 기준)
- 레이팅 구간별·속도별 독립 계산 (1000, 1400, 1800, 2200, 2500)
- Z-score 기반 정규화된 티어 배정

1.2 아키텍처

파일 구조

```
backend/app/features/opening_tier/
├── __init__.py (FastAPI 라우터 등록)
├── openings_catalog.py (ECO 카탈로그: 60+ 오프닝 정의)
├── opening_tips_catalog.py (한국어 핵심 포인트)
├── services/
│   ├── opening_tier_service.py (핵심 알고리즘)
│   └── opening_detail_service.py (상세 정보/YouTube 링크)
└── routes/
    └── [라우터 등록]
```

주요 클래스·함수

항목	파일	역할
`OpeningTierService`	`opening_tier_service.py`	데이터 수집, 티어 계산, 캐시 관리
`get_opening_tiers()`	〃	공개 API: 특정 레이팅/속도의 오프닝 티어 목록 반환
`refresh_cache_for_all()`	〃	수동 갱신: 모든 레이팅×속도 조합 재계산
`load_cache_from_disk_if_valid()`	〃	시작 시 디스크 캐시 로드
`start_midnight_cache_refresher()`	〃	매월 1일 UTC 00:00 자동 갱신 스케줄
`OPENINGS_CATALOG`	`openings_catalog.py`	카탈로그: ECO, 이름, UCI 수열
`generate_fen()`	`openings_catalog.py`	UCI 수열 → FEN 변환

1.3 데이터 수집 프로세스

1.3.1 수집 대상 기간

```
오늘 = 2026-04-03 (UTC)
→ 전월: 2026-03-01 ~ 2026-03-31
```

매월 1일 자정에 **이전 달 전체 데이터**를 대상으로 수집합니다.

1.3.2 카탈로그 기반 탐색 (병렬)

입력: `OPENINGS_CATALOG` (60+ 오프닝)

단계:

1. **FEN 생성**
   각 오프닝의 UCI 수열(예: `["e2e4", "e7e5"]`)을 `python-chess`로 재생하여 **FEN 문자열** 생성
   → 오프닝 포지션의 정확한 표현

2. **병렬 API 호출**

```
┌──────────────────────────────────────────────
│ Semaphore(MAX_CONCURRENT=3)로 동시성 제한
│ 각 오프닝마다:
│   GET https://explorer.lichess.org/lichess?fen=
{FEN}&speeds=bullet,blitz,...
└──────────────────────────────────────────────
```

쿼리 파라미터:

- `fen`: FEN 문자열
- `speeds`: "bullet", "blitz", "rapid", "classical"
- `rated`: true (정식 게임만)
- `since`/`until`: YYYY-MM 형식 (전월)
- `ratings`: "1000,1200,1400,1600,1800,2000,2200,2500"

3. **응답 처리**

```
{
  "moves": [
    {
      "uci": "e5g6",
      "san": "Ng6",
      "white": 45000,
      "draws": 32000,
      "black": 23000,
      "game": 100000
    },
    ...
  ]
}
```

수집 통계: 각 다음 수마다 `(white_wins, draws, black_wins, total_games)` 축적

4. **전개 깊이 제한**
   기본 5-8반(ply) 정도로 제한; `MAX_DEPTH=10` 이상은 수집하지 않음
   → 메모리·시간 효율

1.3.3 BFS 폴백

카탈로그 탐색 결과가 **`MIN_CATALOG_RESULTS=10` 미만**인 경우:

1. 카탈로그 기준점(마지막 FEN)에서 출발
2. **BFS(너비 우선 탐색)**: 상위 `TOP_N_MOVES=5` 변형만 추가 탐색
3. 슬롯 내 요청 간격: `CATALOG_REQUEST_DELAY=0.3초`
4. 최대 노드: `MAX_NODES=200` 또는 깊이 제한

목표: 인기 오프닝은 충분한 데이터를 확보하고, 희귀 변형도 기본 통계 제공

1.4 티어 배정 알고리즘 (Z-score 기반)

1.4.1 점수 계산 공식

각 오프닝(이동)에 대해:

```
점수 = WIN_WEIGHT × 정규화_승률 + POP_WEIGHT × 정규화_인기도

여기서:
  정규화_승률 = (승률 - 전체평균) / 표준편차
  정규화_인기도 = log(게임수 + 1) / log(최대게임수 + 1)
```

상수:

- `WIN_WEIGHT = 0.5` (상대적 성과)
- `POP_WEIGHT = 0.5` (인기도)
- `SCORING_PRIOR = 600` (베이지안 평활화 강도)

1.4.2 승률 계산 (평활화)

```
승률(평활화) = (백승 + SCORING_PRIOR × 0.5) / (총게임수 + SCORING_PRIOR)
```

예시:

- 게임 100개, 백 60승, 흑 30승, 무승부 10: `(60 + 300) / (100 + 600) = 60%`
- 게임 10개, 백 7승, 흑 2승, 무 1: `(7 + 300) / (10 + 600) = 51%` (평활)

목표: 게임 수가 적을 때 극단적 값 방지

1.4.3 색상별(Opening Side) 필터링

ECO 코드 범위 기반 색상 추론:

```
A01~A09, A45~A46 등: 백 오프닝 (화이트가 주도)
A50~A99, B00~B99 등: 주로 흑 방어 (블랙 응답)
```

규칙: 티어 배정 전에 색상 필터를 적용
→ "Caro-Kann Defense"는 **Black 오프닝**으로만 나열, White 목록에는 포함 안 함

1.4.4 Z-score 정규화 및 티어 배정

```
표준정규분포로 변환:
  z = (점수 - 평균) / 표준편차

티어 배정:
  S: z >= 1.0 (상위 15.9%)
  A: 0.5 <= z < 1.0 (상위 30.9%)
  B: 0 <= z < 0.5 (상위 50%)
  C: -0.5 <= z < 0 (하위 50%)
  D: z < -0.5 (하위 30.9%)
```

특성:

- 절대 강도가 아닌 **상대적 순위** 반영
- 생태 변화 자동 반영 (매월 갱신 시)
- 레이팅×속도 조합별 독립 계산 (예: 1400 Blitz는 2200 Rapid과 다름)

1.5 캐싱 및 갱신

1.5.1 캐시 구조

```
_tier_cache = {
    "1400|blitz|white": [  # 캐시 키: "레이팅|속도|색상"
        {
            "eco": "B20",
            "name": "Sicilian Defense",
            "tier": "A",
            "white_wins": 150000,
            "draws": 100000,
            "black_wins": 120000,
            "tier_score": 0.82,
            ...
        },
        ...
    ],
    "1400|blitz|black": [...],
    ...
}
```

저장소:

1. **메모리**: `_tier_cache` (서버 런타임)
2. **디스크**: `backend/data/opening_tier_cache/opening_tier_latest.json`
3. **Supabase Storage**: 인스턴스 간 공유 (선택적)

1.5.2 갱신 주기

```
매월 1일 UTC 00:00
  ↓
refresh_cache_for_all() 호출 (모든 레이팅×속도 조합)
  ↓
5~10분 후 완료
  ↓
캐시 저장 (메모리 + 디스크 + Supabase)
```

자동화: `start_midnight_cache_refresher()` 서버 시작 시 호출

1.5.3 캐시 유효성 검사

```
logic_version (코드 변경 감지)
  ↓
items (컨텐츠)
since / until (데이터 범위)
stamp (생성 날짜)
```

로직 버전이 달라지면 → 기존 캐시 무시, 새로 수집

1.6 API 엔드포인트

GET `/api/v1/opening-tier/tiers/{rating}/{speed}/{color}`

```
curl "http://localhost:8000/api/v1/opening-tier/tiers/1400/blitz/white"
```

응답:

```
{
  "tiers": [
    {
      "eco": "E60",
      "name": "King's Indian Defense",
      "tier": "A",
      "white_wins": 85000,
      "draws": 72000,
      "black_wins": 63000,
      "total_games": 220000,
      "win_rate": 38.6,
      "draw_rate": 32.7,
      "tier_score": 0.78
    },
    ...
  ],
  "data_period": "2026-03-01 ~ 2026-03-31",
  "cache_updated": "2026-04-01"
}
```

파라미터:

- `rating`: 1000, 1400, 1800, 2200, 2500
- `speed`: bullet, blitz, rapid, classical
- `color`: white, black

1.7 성능 특성

항목	값	비고
카탈로그 크기	60+ 오프닝	A~E ECO 시리즈
API 호출당 시간	0.3~2초	병렬화로 전체 5~10분
캐시 크기	~5MB	360KB × 14 조합 (5 레이팅 × 4 속도 × 색상)
메모리 오버헤드	~20MB	메모리 내 모든 데이터 로드
조회 응답 시간	<100ms	메모리 캐시 직접 조회

---

2. 전적 검색의 게임 분석 로직

2.1 개요

Foresight의 **게임 분석** 시스템은 **Stockfish 엔진**을 사용하여 각 게임의 모든 수를 평가하고, **T1(Brilliant) ~ T6(Blunder)** 등급으로 분류합니다. 사용자는 SSE(Server-Sent Events)를 통해 **실시간 스트리밍**으로 경과를 확인할 수 있습니다.

핵심 특징:

- 반수 단위 `cp_loss`, 승률 손실(WPL) 기반 등급 분류
- Stockfish 3초 분석(또는 깊이 제한)
- SSE 스트리밍: 수 하나 완료할 때마다 프론트엔드로 즉시 전송
- 인메모리 LRU 캐싱: 1시간 TTL, 50개 최대 항목

2.2 아키텍처

파일 구조

```
backend/app/
├── ml/
│   ├── game_analyzer.py (핵심 분석 알고리즘)
│   ├── move_classifier.py (수 품질 분류)
│   └── engine.py (Stockfish 래퍼)
├── api/routes/
│   └── game_analysis.py (SSE 엔드포인트, 캐싱, 스케줄링)
└── models/schemas.py (데이터 모델)
```

주요 클래스·함수

항목	파일	역할
`analyze_game_streaming()`	`game_analyzer.py`	진입점: PGN → 모든 수 분석 (콜백 기반)
`_analyze_single_move_with_fen()`	〃	반수 단위: CP 손실, 티어 결정

항목	파일	역할
`_determine_tier()`	〃	임계값 기반 T1~T6 배정
`_get_top_moves()`	〃	MultiPV: 상위 3 대안수 탐색
`analyze_games_sync()`	`move_classifier.py`	여러 게임 집계: 정확도, 수 품질 통계
`classify_move()`	〃	WPL → "Best", "Excellent", ... 분류
`POST /api/v1/game-analysis/game/stream`	`game_analysis.py`	HTTP 엔드포인트

데이터 모델

`AnalyzedMove` (반수 분석 결과):

```
class AnalyzedMove:
    halfmove: int                # 반수 번호 (0부터 시작)
    move_number: int             # 수 번호 (1, 2, 3, ... 흑수는 1.5, 2.5, ...)
    color: str                   # "white" 또는 "black"
    san: str                     # Standard Algebraic Notation (예: "e4",
"Nf3")
    uci: str                     # UCI 표기 (예: "e2e4")
    fen_before: str              # 수 전 FEN
    fen_after: str               # 수 후 FEN
    cp_before: Optional[int]     # 수 전 엔진 평가값 (센티폰 단위)
    cp_after: Optional[int]      # 수 후 엔진 평가값
    cp_loss: int                 # CP 손실 (음수 = 개선)
    win_pct_before: float        # 수 전 승률 (%)
    win_pct_after: float         # 수 후 승률
    win_pct_loss: float          # 승률 손실 (%)
    tier: MoveTier               # T1, T2, TF, TH, ...
    top_moves: List[TopMoveInfo] # 상위 3 대안수
    user_move_rank: int          # 사용자 수의 엔진 순위 (1 = 최고)
    is_only_best: bool           # 유일한 최선수인지
    best_gap_cp: int             # 최고수와의 CP 차
    cp_swing: int                # 수로 인한 포지션 변화량
    is_sacrifice: bool           # 희생인지
    sacrifice_value: int         # 희생 가치 (CP 손실)
```

`PlayerAnalysisResult` (한 색의 요약):

```
class PlayerAnalysisResult:
    color: str                   # "white" 또는 "black"
    username: str                # 플레이어 이름
    total_moves: int             # 수 개수
    analyzed_moves: List[AnalyzedMove]
```

```
    # 요약 통계 (@property)
    accuracy: float                # 정확도 (%)
    tier_counts: Dict[MoveTier, int]  # 각 티어별 개수
    tier_percentages: Dict[MoveTier, float]  # 각 티어별 %
    avg_cp_loss: float             # 평균 CP 손실
```

2.3 분석 알고리즘 상세

2.3.1 진입점: `analyze_game_streaming()`

```
def analyze_game_streaming(
    pgn_str: str,
    game_id: str = "",
    stockfish_depth: Optional[int] = None,  # None → 시간 기반, 숫자 → 깊이 고정
    on_init: Optional[Callable] = None,     # 플레이어 정보 콜백
    on_move: Optional[Callable] = None,     # 수 분석 완료 콜백
) -> BothPlayersAnalysisResult:
```

단계:

1. **PGN 파싱**

```
game = chess.pgn.read_game(io.StringIO(pgn_str))
board = game.board()
```

2. **초기 정보 계산**

```
- 플레이어: game.headers["White"], game.headers["Black"]
- 총 반수: PGN 변화 추적으로 계산
- 오프닝: _compute_opening_theory() → OpeningDB 조회
```

→ `on_init()` 콜백 호출

3. **Stockfish 엔진 시작**

```
with chess.engine.SimpleEngine.popen_uci(STOCKFISH_PATH) as engine:
    engine.configure({
        "Threads": settings.STOCKFISH_THREADS,  # 기본 2
        "Hash": settings.STOCKFISH_HASH_MB,      # 기본 256MB
    })
    # 각 반수 분석...
```

4. **각 반수 분석 루프**

```
while node.variations:
    move = node.variations[0].move
    analyzed_move, prev_info = _analyze_single_move_with_fen(
        engine, board, move, halfmove, stockfish_depth
    )
    on_move(analyzed_move)
    halfmove += 1
```

5. **종료 및 요약**

```
white_analysis = PlayerAnalysisResult(white_moves)
black_analysis = PlayerAnalysisResult(black_moves)
return BothPlayersAnalysisResult(white_analysis, black_analysis)
```

2.3.2 반수 분석: `_analyze_single_move_with_fen()`

목적: 한 반수에 대해 `cp_loss`, 티어, 상대 수 등을 계산

2.3.2.1 강제수 감지 (TF)

```
def _is_forced_move(board: chess.Board, user_move: chess.Move) -> bool:
    # 합법수가 1개뿐이면 강제수(TF)
    if board.legal_moves.count() == 1:
        return True

    # 체크 상황에서 체크를 해제하는 수가 유저 수 하나뿐이면 TF
    if board.is_check():
        other_moves = [m for m in board.legal_moves if m != user_move and
_is_safe(m)]
        if not other_moves:
            return True

    return False
```

2.3.2.2 CP 손실 계산

```
수 전 평가: cp_before (직전 반수의 cp_after 재사용)
수 후 평가: cp_after = engine.analyse(board_after)

cp_loss = cp_after - cp_before
  (양수 = 악화, 음수 = 개선)
```

특수 케이스:

- **Mate**: `cp_loss` = 매트까지 수 차이 (음의 매트 값)
- **Checkmate Avoidance**: 이전 포지션에서 이미 메이트 예정 → cp_loss 무시

2.3.2.3 승률 손실 (Win Pct Loss, WPL)

Lichess 엔진 평가를 Bayesian Elo 모델로 변환:

```
win_pct_before = 100 / (1 + 10^(-cp_before/400))
win_pct_after = 100 / (1 + 10^(-cp_after/400))

wpl = win_pct_before - win_pct_after
```

메이트 우위 구간 안정화:

- `cp >= 9500` (메이트 근처): win_pct_loss 과도 팽창 억제
- 정확도 계산용으로만 완화, 티어 분류는 원래 값 사용

2.3.2.4 상위 대안수 탐색 (MultiPV)

```
def _get_top_moves(board: chess.Board, n: int = 3, depth: Optional[int] =
None) -> List[...]:
    # MultiPV 모드로 상위 n개 수 탐색
    info_list = engine.analyse(
        board,
        limit,
        multipv=n
    )

    result = [
        {
            "uci": move.uci(),
            "san": board.san(move),
            "cp": cp_value,
            "rank": rank
        }
        for rank, (move, cp_value) in enumerate(info_list, start=1)
    ]
    return result
```

2.3.3 티어 배정: `_determine_tier()`

임계값 (CP loss 및 WPL):

티어	CP Loss (max)	WPL % (max)	설명

티어	CP Loss (max)	WPL % (max)	설명
TF	-	-	강제수 (합법수 1개 또는 체크 회피 유일)
TH	-	-	오프닝 이론수 (OpeningDB 일치)
T1	8	1.2	기가급 (Brilliant)
T2	10	1.5	최상급 (Best)
T3	25	4.0	우수 (Excellent)
T4	55	7.5	양호 (Good)
T5	95	14.0	부정확 (Inaccuracy)
T6	140+	22.0+	실수 (Mistake/Blunder)

결정 과정:

```
def _determine_tier(cp_loss, wpl, rank, is_only_best):
    # 우선순위순:
    if is_forced:
        return MoveTier.TF
    if is_theory_match:
        return MoveTier.TH
    if is_t1_candidate and rank == 1 and is_only_best:
        return MoveTier.T1

    for tier, cp_max, wpl_max in TIER_THRESHOLDS:
        if cp_loss <= cp_max and wpl <= wpl_max:
            return tier

    return MoveTier.T6  # 기본값
```

T1 (Brilliant) 추가 조건:

```
1. rank == 1 (최고 수)
2. is_only_best == True (2위와의 차이 >= 40 CP)
3. cp_loss <= T1_MAX_CP_LOSS (8)
4. wpl <= T1_MAX_WIN_PCT_LOSS (1.2%)
5. 전술적 품질 점수 >= 60점
   (CP swing, 결정적 우위 검증 등)
```

2.4 정확도 계산

```
def _compute_accuracy(moves: List[AnalyzedMove]) -> float:
    """
    평균 승률 손실에서 Lichess 방식 정확도 계산.
```

```
    TH(이론수) 제외.
    """
    valid_moves = [m for m in moves if m.tier != MoveTier.TH]
    if not valid_moves:
        return 100.0

    avg_wpl = sum(m.win_pct_loss for m in valid_moves) / len(valid_moves)

    # Lichess 공식 (Bayes 조화평균)
    accuracy = 100 / (1 + avg_wpl / 50)
    return round(accuracy, 1)
```

2.5 SSE 스트리밍 및 캐싱

2.5.1 엔드포인트

```
POST /api/v1/game-analysis/game/stream
Content-Type: application/json

{
  "pgn": "1. e4 e5 2. Nf3 ...",
  "game_id": "12345",
  "stockfish_depth": null  // 또는 20 (깊이 고정)
}
```

응답: `text/event-stream` (SSE)

2.5.2 이벤트 타입

타입	설명	예
`queued`	분석 대기 신호	`{"type": "queued"}`
`init`	필수 메타 정보	`{"type": "init", "total_moves": 40, "white_player": "..."}`
`move`	반수 분석 완료	`{"type": "move", "data": {...AnalyzedMove...}}`
`complete`	전체 완료 + 요약	`{"type": "complete", "white": {...}, "black": {...}}`
`error`	오류 발생	`{"type": "error", "message": "..."}`

예시 스트림:

```
data: {"type": "queued"}

data: {"type": "init", "total_moves": 40, "white_player": "Magnus", ...}
```

```
data: {"type": "move", "data": {"halfmove": 0, "san": "e4", "cp_before":
0, ...}}

data: {"type": "move", "data": {"halfmove": 1, "san": "e5", ...}}

... 40개의 move 이벤트 ...

data: {"type": "complete", "white": {"accuracy": 85.3, "tier_counts":
{...}}, "black": {...}}
```

2.5.3 인메모리 캐시

구조:

```
_analysis_cache: OrderedDict[
    Tuple[str, int],  # (uid, stockfish_depth)
    _StreamCacheEntry
] = OrderedDict()

class _StreamCacheEntry:
    init_event: Dict        # init 이벤트
    move_events: List[Dict] # 모든 move 이벤트 배열
    complete_event: Dict    # complete 이벤트
    timestamp: float        # 저장 시각
```

정책:

항목	값
TTL	3600초 (1시간)
최대 개수	50개
제거 정책	LRU (가장 오래된 항목 제거)
동기화	`threading.Lock`
저장 시점	분석 성공 후 `complete_event` 큐 추가 직후

캐시 키:

```
(game_id, stockfish_depth)
  여기서 game_id가 빈 문자열이면:
  (sha1(pgn)[:16], stockfish_depth) 사용
```

히트 시:

```
큐 대기(세마포어) 제외
→ 저장된 init, move[], complete 즉시 재생
→ 응답 시간 <100ms
```

2.6 동시성 관리

2.6.1 `STOCKFISH_CONCURRENT` 설정

```
STOCKFISH_CONCURRENT = int(os.getenv("STOCKFISH_CONCURRENT", "0"))
```

- **0 (기본)**: 세마포어 없음 → 유저끼리 대기 없이 즉시 분석 시작
- **1 이상**: 그 개수만큼 동시 분석 허용, 나머지는 대기

운영 팁:

- 단일 vCPU 머신: `STOCKFISH_CONCURRENT=1` 권장
- 멀티 vCPU: `0` (HTTP 스케일로 로드 분산)

2.6.2 요청 처리 순서

```
1. PGN 검증
2. 캐시 조회
   - 히트: Semaphore 없음, 즉시 재생
   - 미스: Semaphore 대기 → 후 분석 시작
3. ThreadPoolExecutor에서 분석 실행
   (비동기 SSE 제너레이터에서 queue.Queue로 이벤트 수신)
4. 클라이언트 끊김 감지 (is_disconnected) → 취소
```

2.7 성능 특성

항목	값	비고
엔진 분석 시간	3초/반수 (시간 제한)	또는 depth 고정 시 가변
전체 게임 (40반)	~2분	depth=20 기준
캐시 히트 응답	<100ms	메모리 재생
메모리 (분석 중)	~200MB	Stockfish + 보드 상태
캐시 메모리	~10MB	50개 × ~200KB

---

3. 전적 검색의 유저 분석 로직

3.1 개요

Dashboard 분석 시스템은 사용자의 최근 게임들을 종합적으로 분석하여 다음 정보를 시각화합니다:

1. **첫 수 선호도 및 승률** (e4, d4, c4 등)
2. **오프닝 트리** (깊이별 분기, 색상별)
3. **오프닝별 성과** (베스트/워스트)
4. **결과 트렌드** (시간순 승패 그래프)
5. **상대 분석** (특정 상대의 전적 통계)

핵심 특징:

- Chess.com, Lichess 양쪽 지원
- 페이지 당 300~5000개 게임 분석
- 다층 필터: time_class, 날짜 범위, 색상
- 계층형 오프닝 구조 (기본명 → 변형 → ECO)

3.2 아키텍처

파일 구조

```
backend/app/features/dashboard/
├── services/
│   ├── analysis.py (AnalysisService: 핵심 통계 계산)
│   ├── tactical_analysis.py (전술 패턴 분석)
│   └── [기타 서비스]
├── routes.py (API 라우터)
└── schemas.py (응답 모델)

backend/app/api/routes/
└── stats.py (통합 분석 엔드포인트)
```

주요 클래스·함수

항목	파일	역할
`AnalysisService`	`analysis.py`	핵심 분석 로직
`build_rows()`	〃	GameSummary[] → dict[] 변환
`get_performance_summary()`	〃	전체 요약 통계
`get_opening_stats()`	〃	오프닝별 상-N 통계
`get_first_move_stats()`	〃	첫 수 선호도 분석
`get_opening_tree()`	〃	계층형 오프닝 트리 구성
`get_result_trend()`	〃	시간순 결과 추세

3.3 데이터 수집 단계

3.3.1 게임 조회

```
# Chess.com
games = await chessdotcom_svc.get_recent_games(
    username,
    max_games=300,  # 기본
    time_class="blitz",
    since_ts=None,
    until_ts=None
)

# Lichess
games = await lichess_svc.get_recent_games(
    username,
    max_games=300,
    time_class="blitz",
    since_ms=None,
    until_ms=None
)
```

필드 (GameSummary):

- `game_id`: 고유 ID
- `white`, `black`: 플레이어 이름
- `result`: win/loss/draw
- `time_class`: bullet/blitz/rapid/classical
- `opening_eco`, `opening_name`: 오프닝 정보
- `pgn`: (선택) 게임 기보
- `played_at`: 게임 시각 (ISO8601)

3.3.2 데이터 정규화

```
def build_rows(games: List[GameSummary]) -> List[dict]:
    """GameSummary를 분석용 dict로 변환"""
    rows = []
    for g in games:
        # PGN에서 첫 수 추출
        white_move1, black_move1 = _extract_first_moves(g.pgn or '')

        rows.append({
            "game_id": g.game_id,
            "platform": g.platform.value,  # "chess.com" 또는 "lichess"
            "white": g.white,
            "black": g.black,
            "result": g.result.value,
            "time_class": g.time_class,
            "opening_eco": g.opening_eco or "Unknown",
```

```
            "opening_name": g.opening_name or "Unknown",
            "played_at": g.played_at,
            "white_move1": white_move1,  # "e4", "d4", "c4", ...
            "black_move1": black_move1,  # "e5", "d5", ...
            "url": g.url,
        })
    return rows
```

3.4 분석 로직 상세

3.4.1 전체 요약: `get_performance_summary()`

```
def get_performance_summary(
    username: str,
    platform: Platform,
    games: List[GameSummary],
    time_class: str = "blitz"
) -> PerformanceSummary:
    rows = self.build_rows(games)

    # time_class 필터 (이미 get_recent_games에서 필터됨)
    if rows:
        rows = [r for r in rows if r["time_class"] == time_class]

    total = len(rows)
    wins = sum(1 for r in rows if r["result"] == "win")
    losses = sum(1 for r in rows if r["result"] == "loss")
    draws = sum(1 for r in rows if r["result"] == "draw")

    win_rate = round(wins / total * 100, 1) if total else 0.0
    top_openings = self.get_opening_stats(rows, top_n=5)

    return PerformanceSummary(
        username=username,
        platform=platform,
        time_class=time_class,
        total_games=total,
        wins=wins,
        losses=losses,
        draws=draws,
        win_rate=win_rate,
        top_openings=top_openings,
    )
```

응답 예:

```
{
  "username": "Magnus",
```

```
  "platform": "chess.com",
  "time_class": "blitz",
  "total_games": 150,
  "wins": 95,
  "losses": 35,
  "draws": 20,
  "win_rate": 63.3,
  "top_openings": [
    {
      "eco": "E60",
      "name": "King's Indian Defense",
      "games": 25,
      "wins": 18,
      "losses": 5,
      "draws": 2,
      "win_rate": 72.0
    },
    ...
  ]
}
```

3.4.2 오프닝별 통계: `get_opening_stats()`

```
def get_opening_stats(
    rows: List[dict],
    top_n: int = 10
) -> List[OpeningStats]:
    """오프닝별로 게임 수, 성과 통계 집계"""

    # ECO + 풀 이름을 키로 사용
    stats: Dict[(str, str), Dict] = defaultdict(lambda: {"win": 0, "loss":
0, "draw": 0})

    for row in rows:
        key = (row["opening_eco"], row["opening_name"])
        result = row["result"]
        stats[key][result] += 1

    # 통계 계산
    result_list = []
    for (eco, name), counts in stats.items():
        wins = counts.get("win", 0)
        losses = counts.get("loss", 0)
        draws = counts.get("draw", 0)
        total = wins + losses + draws

        if total == 0:
            continue

        result_list.append(
```

```
            OpeningStats(
                eco=eco,
                name=name,
                games=total,
                wins=wins,
                losses=losses,
                draws=draws,
                win_rate=round(wins / total * 100, 1),
            )
        )

    # 게임 수로 정렬 + 상위 N개
    result_list.sort(key=lambda x: x.games, reverse=True)
    return result_list[:top_n]
```

3.4.3 첫 수 분석: `get_first_move_stats()`

```
def get_first_move_stats(
    rows: List[dict],
    username: str
) -> Dict[str, Any]:
    """
    백/흑 첫 수 선호도 분석
    - 사용자가 백일 때: white_move1 기준 집계
    - 사용자가 흑일 때: black_move1 기준 집계
    - 최소 5게임 이상인 수만 포함
    """
    if not rows:
        return {"white": [], "black": [], "total_games": 0}

    username_lower = username.lower()

    # 색상별로 필터
    white_rows = [r for r in rows if r.get("white", "").lower() ==
username_lower]
    black_rows = [r for r in rows if r.get("black", "").lower() ==
username_lower]

    def first_move_stats(subset: List[dict], move_col: str, top_n: int =
7):
        if not subset:
            return []

        # 첫 수별로 집계
        agg: Dict[str, Dict] = defaultdict(lambda: {"win": 0, "loss": 0,
"draw": 0})
        for row in subset:
            move = row.get(move_col)
            if not move:
                continue
```

```
            result = row["result"]
            agg[move][result] += 1

        # 필터 + 통계 계산
        result = []
        for move, counts in agg.items():
            wins = int(counts.get("win", 0))
            losses = int(counts.get("loss", 0))
            draws = int(counts.get("draw", 0))
            total = wins + losses + draws

            # 최소 5게임 이상
            if total < 5:
                continue

            result.append({
                "eco": move,                    # "e4", "d4" 등
                "first_move_category": move,
                "games": total,
                "wins": wins,
                "losses": losses,
                "draws": draws,
                "win_rate": round(wins / total * 100, 1),
            })

        # 게임 수로 정렬 + 상위 N
        result.sort(key=lambda x: x["games"], reverse=True)
        return result[:top_n]

    return {
        "white": first_move_stats(white_rows, "white_move1"),
        "black": first_move_stats(black_rows, "black_move1"),
        "total_games": len(rows),
    }
```

응답 예:

```
{
  "white": [
    {
      "eco": "e4",
      "first_move_category": "e4",
      "games": 45,
      "wins": 32,
      "losses": 10,
      "draws": 3,
      "win_rate": 71.1
    },
    {
      "eco": "d4",
      "games": 30,
      "wins": 18,
```

```
      "losses": 9,
      "draws": 3,
      "win_rate": 60.0
    },
    ...
  ],
  "black": [
    {
      "eco": "e5",
      "games": 40,
      "wins": 25,
      "losses": 12,
      "draws": 3,
      "win_rate": 62.5
    },
    ...
  ],
  "total_games": 150
}
```

3.4.4 오프닝 트리: `get_opening_tree()`

```
def get_opening_tree(
    rows: List[dict],
    depth: int = 3
) -> List[dict]:
    """
    계층형 오프닝 트리 구성:

    Level 1: 기본 오프닝명 (콜론 앞 부분)
             예) "Caro-Kann Defense"
    Level 2: 풀 변형명 + ECO 코드
             예) "Caro-Kann Defense: Advance Variation" (B12)

    각 노드는 게임 수, 승률 포함
    """
    if not rows:
        return []

    base_tree: Dict[str, Dict] = {}

    for row in rows:
        eco = row.get("opening_eco", "") or ""
        name = row.get("opening_name", "") or ""
        result = row.get("result", "draw")

        # 기본 오프닝명 추출
        if ":" in name:
            base_name = name.split(":", 1)[0].strip()
        elif name and name != "Unknown":
```

```
            base_name = name.strip()
        else:
            # OpeningDB에서 ECO 기반 조회
            base_name = opening_db.get_name_by_eco(eco[:3]) if eco else
"Unknown"
            if not base_name:
                base_name = eco[:1] + "xx" if eco else "Unknown"

        # 트리 구조 구성
        if base_name not in base_tree:
            base_tree[base_name] = {
                "name": base_name,
                "games": 0,
                "wins": 0,
                "losses": 0,
                "draws": 0,
                "variations": {}
            }

        base_node = base_tree[base_name]
        base_node["games"] += 1
        base_node[result] += 1

        # 변형명 추가
        variation_key = f"{eco} {name}"
        if variation_key not in base_node["variations"]:
            base_node["variations"][variation_key] = {
                "eco": eco,
                "name": name,
                "games": 0,
                "wins": 0,
                "losses": 0,
                "draws": 0
            }

        var_node = base_node["variations"][variation_key]
        var_node["games"] += 1
        var_node[result] += 1

    # 승률 계산 + 트리 구조화
    result = []
    for base_name, base_data in base_tree.items():
        total = base_data["games"]
        base_wr = round(base_data["wins"] / total * 100, 1) if total else
0

        variations = []
        for var_key, var_data in base_data["variations"].items():
            var_total = var_data["games"]
            var_wr = round(var_data["wins"] / var_total * 100, 1) if
var_total else 0

            variations.append({
                "eco": var_data["eco"],
```

```
                "name": var_data["name"],
                "games": var_total,
                "win_rate": var_wr
            })

        variations.sort(key=lambda x: x["games"], reverse=True)

        result.append({
            "base_name": base_name,
            "games": total,
            "win_rate": base_wr,
            "variations": variations[:depth]  # 상위 depth개만
        })

    result.sort(key=lambda x: x["games"], reverse=True)
    return result
```

응답 예:

```
[
  {
    "base_name": "King's Indian Defense",
    "games": 45,
    "win_rate": 68.9,
    "variations": [
      {
        "eco": "E91",
        "name": "King's Indian Defense: Classical",
        "games": 20,
        "win_rate": 70.0
      },
      {
        "eco": "E60",
        "name": "King's Indian Defense",
        "games": 15,
        "win_rate": 66.7
      },
      {
        "eco": "E80",
        "name": "King's Indian Defense: Samisch",
        "games": 10,
        "win_rate": 70.0
      }
    ]
  },
  ...
]
```

3.4.5 결과 트렌드: `get_result_trend()`

```python
def get_result_trend(rows: List[dict]) -> List[dict]:
    """시간순 승/패/무 트렌드 (프론트 차트용)"""
    if not rows:
        return []

    # 시간순 정렬
    sorted_rows = sorted(rows, key=lambda r: r["played_at"])

    return [
        {
            "played_at": r["played_at"],
            "win": 1 if r["result"] == "win" else 0,
            "loss": 1 if r["result"] == "loss" else 0,
            "draw": 1 if r["result"] == "draw" else 0,
        }
        for r in sorted_rows
    ]
```

응답 예:

```json
[
  {
    "played_at": "2026-04-01T10:30:00Z",
    "win": 1,
    "loss": 0,
    "draw": 0
  },
  {
    "played_at": "2026-04-01T10:35:00Z",
    "win": 0,
    "loss": 1,
    "draw": 0
  },
  ...
]
```

3.5 API 엔드포인트

GET `/api/v1/stats/first-moves/{platform}/{username}`

```bash
curl "http://localhost:8000/api/v1/stats/first-moves/chess.com/Magnus?
time_class=blitz&max_games=300"
```

파라미터:

- `platform`: chess.com, lichess

- `username`: 플레이어 이름
- `time_class`: bullet, blitz, rapid, classical
- `max_games`: 1~5000 (기본 300)
- `since_ms`, `until_ms`: Unix ms 기반 날짜 필터 (선택)

GET `/api/v1/stats/opening-tree/{platform}/{username}`

```
curl "http://localhost:8000/api/v1/stats/opening-tree/chess.com/Magnus?
time_class=blitz&depth=3&side=white"
```

파라미터:

- `side`: white, black (특정 색 필터, 선택)
- `depth`: 1~5 (변형명 깊이, 기본 3)

GET `/api/v1/stats/opening-best-worst/{platform}/{username}`

```
curl "http://localhost:8000/api/v1/stats/opening-best-
worst/chess.com/Magnus?min_games=10"
```

응답: 최고/최악 오프닝 각 N개

3.6 성능 특성

항목	값	비고
게임 조회	200~1s (300건)	API 호출 대기
데이터 정규화	50ms	메모리 내 처리
통계 계산	100~500ms	크기 및 필터에 따라
응답 시간	<1.5s	캐시 미적용 기준
캐싱	staleTime 5분, gcTime 30분	React Query (프론트)

---

4. 데이터베이스 구조

4.1 개요

Foresight는 **PostgreSQL**을 기반으로 다음 주요 엔티티를 관리합니다:

- **User**: 계정 정보, OAuth 통합
- **Post**: 포럼 글
- **Comment**: 댓글 및 대댓글

- **PostLike**: 좋아요

게임 데이터(전적, 분석)는 **외부 API** (Chess.com, Lichess)에서 직접 조회하며, 디스크 캐싱만 사용합니다.

4.2 스키마 정의

4.2.1 `users` 테이블

```sql
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    public_id VARCHAR(21) NOT NULL UNIQUE,           -- 공개 프로필 URL용

    -- OAuth 통합
    provider VARCHAR(32) NOT NULL,                   -- "github",
"google", "lichess", ...
    provider_account_id VARCHAR(255) NOT NULL,       -- 외부 ID
    UNIQUE (provider, provider_account_id),

    -- 프로필
    email VARCHAR(320),
    display_name VARCHAR(255) NOT NULL DEFAULT '',
    avatar_url VARCHAR(2048),
    avatar_sync_oauth BOOLEAN NOT NULL DEFAULT TRUE, -- OAuth에서 자동 갱신

    -- 권한 및 상태
    role VARCHAR(20) NOT NULL DEFAULT 'user',        -- "user",
"moderator", "admin"
    signup_completed BOOLEAN NOT NULL DEFAULT FALSE,
    terms_accepted_at TIMESTAMP WITH TIME ZONE,
    profile_public BOOLEAN NOT NULL DEFAULT TRUE,

    -- 이메일 인증
    email_verified_at TIMESTAMP WITH TIME ZONE,
    signup_email_code_hash VARCHAR(64),
    signup_email_code_expires_at TIMESTAMP WITH TIME ZONE,
    signup_email_verify_attempts INTEGER NOT NULL DEFAULT 0,

    -- 타임스탬프
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_users_provider ON users(provider);
CREATE INDEX idx_users_public_id ON users(public_id);
```

주요 필드:

- `id`: SQLAlchemy ORM 트래킹용 PK
- `public_id`: URL-safe 21자 ID (nanoid 형식)
- `provider`, `provider_account_id`: OAuth 매핑
- `avatar_sync_oauth`: True일 때 로그인 시마다 OAuth에서 avatar 새로고침

4.2.2 `posts` 테이블

```
CREATE TABLE posts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    public_id VARCHAR(21) NOT NULL UNIQUE,

    -- 작성자 및 내용
    author_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    title VARCHAR(500) NOT NULL,
    body TEXT NOT NULL DEFAULT '',

    -- 체스판 데이터 (선택)
    pgn_text TEXT,                                  -- 게임 기보
    fen_initial VARCHAR(120),                       -- 초기 FEN (문제)
    board_annotations JSONB,                        -- 보드 주석 (JSON)

    -- 타임스탬프
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),

    -- 관리
    is_hidden BOOLEAN NOT NULL DEFAULT FALSE,
    hidden_reason VARCHAR(500),
    hidden_by_id UUID,                              -- 숨긴 모더레이터
    deleted_at TIMESTAMP WITH TIME ZONE,            -- Soft delete
    board_category VARCHAR(20),                     -- "notice",
"patch", "free", NULL(포럼)

    INDEX idx_posts_author ON author_id,
    INDEX idx_posts_created ON created_at,
    INDEX idx_posts_category ON board_category
);
```

주요 필드:

- `pgn_text`: PGN 기보 (체스 일반 공개 표기법)
- `fen_initial`: 초기 FEN 위치 (체스 문제용)
- `board_annotations`: 각 칸별 주석 (예: `{"e4": "very strong", ...}`)
- `board_category`: 글 카테고리 (NULL이면 일반 포럼)
- `deleted_at`: Soft delete 플래그

4.2.3 `comments` 테이블

```
CREATE TABLE comments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    -- 관계
```

```
    post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
    author_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    parent_comment_id UUID REFERENCES comments(id) ON DELETE CASCADE,  --
대댓글

    -- 내용
    body TEXT NOT NULL,

    -- 타임스탬프
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),

    -- 관리
    is_hidden BOOLEAN NOT NULL DEFAULT FALSE,
    deleted_at TIMESTAMP WITH TIME ZONE,

    INDEX idx_comments_post ON post_id,
    INDEX idx_comments_author ON author_id,
    INDEX idx_comments_parent ON parent_comment_id
);
```

특징:

- Self-join (`parent_comment_id`) 대댓글 지원
- Soft delete 구현

4.2.4 `post_likes` 테이블

```
CREATE TABLE post_likes (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    post_id UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),

    UNIQUE (post_id, user_id),
    INDEX idx_post_likes_post ON post_id,
    INDEX idx_post_likes_user ON user_id
);
```

4.3 게임 데이터 스키마 (메모리)

게임 데이터는 **외부 API 응답**을 메모리 모델로 매핑합니다.

4.3.1 `GameSummary` (Pydantic 모델)

```
class GameSummary(BaseModel):
    game_id: str                            # 플랫폼 고유 ID
    platform: Platform                      # Platform.chessdotcom,
Platform.lichess
    white: str                              # 백 플레이어 이름
    black: str                              # 흑 플레이어 이름
    result: GameResult                      # win, loss, draw
    time_class: str                         # bullet, blitz, rapid,
classical

    # 오프닝 정보
    opening_eco: Optional[str] = None       # "B20", "E60", ...
    opening_name: Optional[str] = None      # "Sicilian Defense", ...

    # 선택 필드
    pgn: Optional[str] = None               # 전체 게임 기보
    played_at: Optional[str] = None         # ISO8601 타임스탬프
    url: Optional[str] = None               # 플랫폼 게임 링크
    rating_white: Optional[int] = None
    rating_black: Optional[int] = None

    # 분석 데이터 (선택)
    cp_evals: Optional[List[Optional[float]]] = None
    move_analysis: Optional[List[Any]] = None
```

4.3.2 `AnalyzedMove` (Pydantic 모델)

```
class AnalyzedMove(BaseModel):
    halfmove: int                   # 반수 번호
    move_number: int                # 수 번호
    color: str                      # "white" 또는 "black"
    san: str                        # "e4", "Nf3", ...
    uci: str                        # "e2e4", "g1f3", ...

    # 위치 평가
    fen_before: str
    fen_after: str
    cp_before: Optional[int]
    cp_after: Optional[int]
    cp_loss: int

    # 승률 변화
    win_pct_before: float
    win_pct_after: float
    win_pct_loss: float

    # 분류 및 분석
    tier: MoveTier                  # T1, T2, T3, ..., TF, TH
    top_moves: List[TopMoveInfo]    # 상위 3 대안
```

```
    user_move_rank: int              # 엔진 순위
    is_only_best: bool               # 유일한 최선
    best_gap_cp: int                 # 2위와의 차이

    # 추가 정보
    cp_swing: int                    # 포지션 변화량
    is_sacrifice: bool
    sacrifice_value: int
```

4.4 스키마 관계도

```
┌──────────────┐
│    users     │
├──────────────┤
│ id (PK)      │
│ public_id    │
│ provider     │
│ ...          │
└──────────────┘
       │
       ├──────────────────────┬──────────────────┐
       │                      │                  │
       ▼                      ▼                  ▼
┌──────────────┐      ┌──────────────┐   ┌──────────────┐
│   posts      │      │  comments    │   │ post_likes   │
├──────────────┤      ├──────────────┤   ├──────────────┤
│ id (PK)      │      │ id (PK)      │   │ id (PK)      │
│ author_id    │◄─────┤ author_id    │   │ post_id      │
│ title        │  │   │ post_id      │   │ user_id      │
│ body         │  │   │ parent_...   │   │              │
│ pgn_text     │  └───┤ body         │   │              │
│ fen_initial  │      │              │   │              │
│ ...          │      └──────┬───────┘   └──────────────┘
└──────────────┘             │
                             └────┘
                   (자체 참조: 대댓글)
```

4.5 외부 API 응답 구조

4.5.1 Chess.com API

게임 목록 응답 구조:

```
{
  "games": [
    {
      "url": "https://www.chess.com/live/game/12345678",
```

```
      "pgn": "1. e4 e5 2. Nf3 ...",
      "time_class": "blitz",
      "end_time": 1680000000,
      "rated": true,

      "white": {
        "username": "Magnus",
        "result": "win"  // "win", "loss", "draw", "abandoned"
      },
      "black": {
        "username": "Ding",
        "result": "loss"
      }
    },
    ...
  ]
}
```

오프닝 정보:

- Chess.com API는 기본적으로 오프닝 정보 미제공
- Foresight는 PGN에서 수를 추출하여 **eco3db** (온라인 ODT) 또는 내부 DB로 오프닝 코드 매핑

4.5.2 Lichess API

게임 목록 응답 구조 (NDJSON):

```
{
  "id": "abc123def456",
  "rated": true,
  "variant": "standard",
  "speed": "blitz",
  "perf": "blitz",
  "createdAt": 1680000000000,
  "players": {
    "white": {
      "user": {
        "name": "Magnus"
      },
      "rating": 2500,
      "ratingDiff": +10
    },
    "black": {
      "user": {
        "name": "Ding"
      },
      "rating": 2480,
      "ratingDiff": -12
    }
  },
  "pgn": "1. e4 e5 2. Nf3 ...",
```

```
  "opening": {
    "eco": "B20",
    "name": "Sicilian Defense",
    "ply": 2
  },
  "status": "mate"  // "created", "started", "mate", "resign", ...
}
```

특징:

- Lichess는 `opening` 객체에 ECO·이름 기본 포함
- PGN 분석 옵션 (`evals`=true) 시 `analysis` 배열 추가 가능

4.6 캐싱 레이어

디스크 캐시

Opening Tier 캐시:

```
backend/data/opening_tier_cache/opening_tier_latest.json
```

구조:

```
{
  "logic_version": 9,
  "stamp": "2026-04-01",
  "since": "2026-03-01",
  "until": "2026-03-31",
  "items": {
    "1400|blitz|white": [
      {
        "eco": "B20",
        "name": "Sicilian Defense",
        "tier": "A",
        ...
      }
    ]
  }
}
```

게임 분석 캐시:

메모리 인메모리만, 디스크 미지원

```
_analysis_cache: OrderedDict[(uid, depth), _CacheEntry]
  - TTL: 3600s
```

```
  - LRU: 50개 최대
```

의존성

- **외부 DB**: PostgreSQL/Supabase (사용자, 포럼)
- **외부 API**: Chess.com API, Lichess API (게임 데이터)
- **Object Storage**: Supabase Storage (Opening tier 원본 캐시)
- **Engine**: Stockfish (분석)

4.7 마이그레이션 및 버전 관리

Alembic 마이그레이션

```
backend/alembic/versions/
├── 20250323_0001_forum_initial.py
├── 20260323_0002_signup_moderation.py
├── 20260323_0004_profile_nickname.py
├── 20260324_0005_post_board_category.py
├── 20260325_0006_post_public_id.py
├── 20260326_0007_user_public_id.py
├── 20260327_0008_signup_email_verify.py
├── 20260328_0009_reports_reporter_unique.py
├── 20260329_0010_avatar_sync_oauth.py
├── 20260330_0011_comment_parent_id.py
└── 20260330_0012_post_board_annotations.py
```

적용:

```
# 자동 마이그레이션 (최신)
alembic upgrade head

# 특정 버전으로
alembic upgrade 20260330_0012_post_board_annotations
```

부록

A. 용어 정의

용어	정의
Halfmove	한 색상의 한 수 (예: White e4 = 반수 0, Black e5 = 반수 1)
Ply	Halfmove와 동일 의미
CP Loss	수 전후 Centipawn 차이 (양수=악화)

용어	정의
WPL	Win Percent Loss, 승률 손실(%)
ECO	Encyclopaedia of Chess Openings, 오프닝 코드 (A00~E99)
FEN	Forsyth-Edwards Notation, 체스판 위치 표기
PGN	Portable Game Notation, 게임 기보 표기
Tier	수의 품질 등급 (T1~T6, TF, TH)
MultiPV	Stockfish의 상위 N개 라인 탐색 모드
SSE	Server-Sent Events, 서버에서 클라이언트로의 단방향 실시간 업데이트

B. 외부 참고 자료

- Chess.com API Docs
- Lichess API Docs
- Lichess Explorer API
- Stockfish Documentation
- Python-chess
- FastAPI Documentation

문서 관리: 코드 변경 시 해당 섹션 동시 갱신
최종 검토일: 2026년 4월 3일